

# Agent Skill Firewall

## Source-Code Companion (code.pdf)

This document accompanies the `agent-skill-firewall` pip package. Each section describes one source file: its purpose, the public API, the design choices behind it, and the lessons it embodies for building larger systems. Sidebars marked *Guide for coding large systems* extract those lessons explicitly so the document doubles as a teaching reference.

## Contents

|           |                                                                |          |
|-----------|----------------------------------------------------------------|----------|
| <b>1</b>  | <b>Package layout (one paragraph)</b>                          | <b>2</b> |
| <b>2</b>  | <b>firewall/config.py – the policy lives here</b>              | <b>2</b> |
| 2.1       | What it does . . . . .                                         | 2        |
| 2.2       | Key dataclasses . . . . .                                      | 2        |
| 2.3       | Notable functions . . . . .                                    | 2        |
| <b>3</b>  | <b>firewall/runtime/firewall.py – the L2 verifier</b>          | <b>3</b> |
| 3.1       | What it does . . . . .                                         | 3        |
| 3.2       | Key functions . . . . .                                        | 3        |
| 3.3       | Five invariants . . . . .                                      | 3        |
| <b>4</b>  | <b>firewall/runtime/egress_proxy.py – L1 containment</b>       | <b>4</b> |
| 4.1       | What it does . . . . .                                         | 4        |
| 4.2       | Why this is L1 and not L2 . . . . .                            | 4        |
| <b>5</b>  | <b>firewall/runtime/canary.py – honeypot tripwire</b>          | <b>4</b> |
| <b>6</b>  | <b>firewall/runtime/supervisor.py – log → evidence</b>         | <b>4</b> |
| 6.1       | What it does . . . . .                                         | 4        |
| <b>7</b>  | <b>firewall/runtime/events.py – JSONL emitter</b>              | <b>4</b> |
| 7.1       | What it does . . . . .                                         | 4        |
| <b>8</b>  | <b>firewall/gate/static_scanner.py – L0</b>                    | <b>5</b> |
| 8.1       | What it does . . . . .                                         | 5        |
| <b>9</b>  | <b>firewall/skills/discover.py – framework-agnostic loader</b> | <b>5</b> |
| 9.1       | What it does . . . . .                                         | 5        |
| 9.2       | Why this matters . . . . .                                     | 5        |
| <b>10</b> | <b>firewall/panel/tui.py – live Textual panel</b>              | <b>5</b> |
| <b>11</b> | <b>firewall/cli.py – subcommands</b>                           | <b>5</b> |
| <b>12</b> | <b>Tests (tests/unit/)</b>                                     | <b>6</b> |

|                                                                |   |
|----------------------------------------------------------------|---|
| 13 Docker (firewall/docker/)                                   | 6 |
| 14 Experiment harness (experiment_minimal/)                    | 6 |
| 15 Agent integration commands (firewall hook, firewall doctor) | 6 |

---

## 1 Package layout (one paragraph)

`firewall/` is the public package. `cli.py` is the `firewall` command-line entry point. `config.py` owns the TOML configuration – the firewall’s policy lives here, not in the code, which is what makes the system non-empirical. `runtime/` contains the L2 verifier and the L1 containment substrate (egress proxy + canary + supervisor + events emitter). `gate/` contains the L0 static scanner. `skills/` is the framework-agnostic skill discoverer. `panel/` is the Textual TUI. `docker/` ships the minimal compose stack. `examples/` bundles two didactic skills used by the tests. Tests live in `tests/unit/`; a minimal experiment harness lives in `experiment_minimal/`.

## 2 firewall/config.py – the policy lives here

### 2.1 What it does

Loads, validates, and writes the TOML configuration file at `~/.config/firewall/config.toml` (XDG; macOS uses `~/Library/Application Support/firewall/`). The default config is held as a literal string so a fresh install can run without any external state.

### 2.2 Key dataclasses

- `DriverConfig` – which LLM the runtime calls when it needs a tie-breaker judge. The *name* of the env var is stored in `api_key_env`; the key itself is read at runtime, never written to disk.
- `PolicyConfig` – one of three modes: `strict`, `balanced` (default), `observe`.
- `EgressConfig` – `allow_hosts` and `deny_hosts`, both wildcard-suffix matched (`*.example.com`).
- `SecretsConfig` – filesystem paths and env-var names treated as SECRET sources for taint analysis.
- `JudgeConfig` – a *minimal* regex list of refusal and compliance patterns. The user is expected to extend these in config without touching the source.
- `TaintConfig` – the six pattern lists that drive L2’s classifier (secret sources, network sinks, exec sinks, exfil host substrings, pseudo-TLDs). Every signal the runtime uses is named here.
- `Config` – the top-level container with three convenience methods (`host_allowed`, `host_denied`, `is_hostile_host`) that encapsulate the wildcard match.

### 2.3 Notable functions

```
def load_config(path=None) -> Config:
    """Load TOML from disk, applying defaults for any missing keys."""

def init_default(path=None, *, overwrite=False) -> Path:
    """Write the bundled default TOML if no file exists."""
```

## Guide for coding large systems

**Push policy out of code.** Patterns, thresholds, and allowlists should read from a user-editable file. Code that *decides* should not *also* carry the decision data. Here, the entire TaintConfig block is data; the verifier in runtime/firewall.py only knows how to read it.

## 3 firewall/runtime/firewall.py – the L2 verifier

### 3.1 What it does

Given an intent envelope (what the skill declared) and the evidence from one run (what the skill did), returns an IARVerdict. About 250 lines, policy-free.

### 3.2 Key functions

```
def build_intent_envelope(skill: dict) -> dict:
    """Derive {name, description, allowed_tools, allow_hosts}."""

def analyze_iar(evidence, envelope, cfg, *, honeypot_aware=False) ->
    IARVerdict:
    """Score one run's evidence. Returns a verdict ladder:
        BENIGN      -- in-intent only
        SUSPICIOUS  -- divergence, no confirmation
        MALICIOUS   -- divergence confirmed by harmful result
    """

def recommend_enforcement(verdict) -> dict:
    """Pure data: 'none' / 'tighten_egress' / 'quarantine'."""
```

### 3.3 Five invariants

The verifier preserves five invariants regardless of user config:

1. **Cross-plane taint.** A secret read on the tool-call plane AND an out-of-policy egress on the network plane in the same run = malicious, regardless of how innocent the destination looks alone.
2. **Pre-scan defer.** Only CRITICAL static findings hard-block; HIGH/MEDIUM defer to the L2 layer.
3. **Detection / containment separation.** In detector mode (honeypot\_aware=False), honeypot capture and canary hits are recorded under `containment` but excluded from the verdict. Reaching the honeypot means the detector already failed.
4. **Unknown is not guilty.** An unfamiliar host with no cross-plane signal is classified `low`, not `suspicious`. Only taint can promote it.

5. **IAR three-plane shape.** **intent** (envelope), **action** (tool calls + egress *attempts*, never bodies), **result** (taint flow / containment). The verifier never reads response bodies.

#### Guide for coding large systems

**State your invariants in writing.** If you cannot list the things a security module must do regardless of configuration, it will drift. Invariants belong at the top of the file and in named tests; if a refactor breaks an invariant test, that is the refactor's problem.

## 4 firewall/runtime/egress\_proxy.py – L1 containment

### 4.1 What it does

A tiny HTTP/HTTPS forward proxy. Allowed hosts pass through, denied or hostile hosts get 403, anything else is *captured* (short-circuited with a 204 after the body is recorded as evidence). Every decision is emitted as a JSONL event the supervisor and the panel consume.

### 4.2 Why this is L1 and not L2

Containment captures *do not* promote a verdict on their own; that discipline lives in the verifier (invariant 3). Keeping the proxy "dumb" – it only knows the allow/deny lists in `cfg.egress` – means a user can audit the containment behaviour without reading any detection logic.

#### Guide for coding large systems

**Containment and detection belong in different files.** The egress proxy can be replaced by an nftables rule, an Envoy sidecar, or a host firewall; nothing in `runtime/firewall.py` cares. The events emitter is the seam.

## 5 firewall/runtime/canary.py – honeypot tripwire

A 28-line HTTP server on 127.0.0.1:8088. Any inbound request is logged as a `CANARY-HIT`. The L2 verifier records this under `containment` only.

## 6 firewall/runtime/supervisor.py – log → evidence

### 6.1 What it does

Parses the proxy's and canary's log lines into typed `Event` records, then aggregates a list of events into the dict shape `analyze_iar` expects. Keeps the regex set out of the verifier.

## 7 firewall/runtime/events.py – JSONL emitter

### 7.1 What it does

A thread-safe `EventEmitter` that appends one JSON object per line to `events.jsonl`, with size-based rotation. This is the seam between every L1/L2 component and the panel; new event kinds can be added without breaking readers.

## Guide for coding large systems

**Make one wire format the contract.** The events file is the firewall's wire format. The proxy writes it, the verifier writes it, the panel reads it, and the experiment harness reads it. New components fit in by writing or reading the same JSONL; no other coupling needed.

## 8 `firewall/gate/static_scanner.py` – L0

### 8.1 What it does

A small static scanner with three rules: hidden-instruction HTML comments in `SKILL.md`, shell-fetch lines in code, and hostile-host literals (from `cfg.taint.exfil_host_substrings`). Returns `Finding` records with a severity; never decides install / no-install on its own (the prescan rule lives in the verifier, invariant 2).

## 9 `firewall/skills/discover.py` – framework-agnostic loader

### 9.1 What it does

Walks the directories every agent framework uses for skills (`.claude/skills/`, `.openclaw/skills/`, `.cursor/skills/`, `.opencode/skills/`, `skills/`) and parses each `SKILL.md` into a `Skill` dataclass. Optionally falls back to PyYAML for the manifest body; the built-in parser handles the small subset that real skills use.

### 9.2 Why this matters

The Agent Skill format converges across frameworks: a directory whose name matches a `name` field in a single text manifest. Hooking at the filesystem prefix means the firewall is uniform across Anthropic SDK, Claude Code, OpenClaw, Cursor, OpenCode, etc.

## Guide for coding large systems

**Hook at the lowest common denominator.** If multiple frameworks agree on the file-system layout, that is your interface, not the agent API of any one of them. The number of agents will grow; the file-system semantics will not.

## 10 `firewall/panel/tui.py` – live Textual panel

A four-region grid: passed/blocked table, session stats, API usage, and a tailing log. Reads `events.jsonl` only – no firewall enforcement of its own. Keyboard: `q` quit, `p` pause, `c` clear table.

## 11 `firewall/cli.py` – subcommands

The Typer-based front door. Subcommands: `config init/show/path`, `scan`, `watch`, `start` (`-mode host|docker`), `stop`, `panel`, `skills`. No subcommand carries policy; each delegates to its module.

## 12 Tests (`tests/unit/`)

Four small files:

- `test_firewall_invariants.py` – one test per invariant from the L2 verifier (a sixth test pins the pseudo-TLD escalation).
- `test_skill_discovery.py` – frontmatter parser + isolated-dir discovery.
- `test_supervisor.py` – regex set for the egress / canary log lines.
- `test_static_scanner.py` – the canary example flags the hostile-host literal; the safe example is clean.

### Guide for coding large systems

**One test per invariant, named after it.** A failing test for the cross-plane-taint rule should literally read `test_cross_plane_taint_is_malicious`. Invariant tests double as documentation.

## 13 Docker (`firewall/docker/`)

A minimal image (`python:3.11-slim`, non-root, read-only root-fs in compose) that runs the egress proxy on 8080 and the canary on 8088 via a 14-line `entrypoint.sh`. The compose file ships a commented `openclaw` service stub so the user can wire any agent that obeys `HTTP_PROXY`.

## 14 Experiment harness (`experiment_minimal/`)

Two files: `run.py` loops over a skills folder and prints a CSV of (`skill`, `static_severity`, `runtime_verdict`, `score`, `reason`); `panel.html` is a 90-line browser dashboard that polls `events.jsonl`.

## 15 Agent integration commands (`firewall hook`, `firewall doctor`)

`firewall hook openclaw` prints the env vars + per-provider config commands to route OpenClaw's HTTP egress through the proxy on `127.0.0.1:8080`. Verified against `openclaw 2026.6.10`: `HTTP_PROXY`, `HTTPS_PROXY`, and `ALL_PROXY` are honoured at the undici layer and catch both OpenClaw's own HTTP and any subprocess spawned by skills (`curl`, `pip`, `npm`). For model-provider calls specifically, `models.providers.<id>.request.proxy.mode` and `.proxyUrl` pin the path. The same command supports `claude-code`, `cursor`, and `generic` targets.

`firewall doctor` runs a six-step health check: config parses, the configured API-key env var is set, the proxy and canary ports are free, and the two optional tools (`docker`, `tectonic`) are reachable. Exit status is 0 on success and 1 if any required check fails. Pair it with `firewall config show` when triaging a fresh install.

## Closing notes on system design

Three habits this codebase encodes:

1. **Push policy out of code.** The runtime decides; the config carries the data it decides over. Patterns rot; code that reads them does not.

2. **Separate detection and containment.** They have different failure modes. Mixing them produces a system you cannot audit when the user disagrees with a verdict.
3. **One wire format is the contract.** An append-only JSONL is the seam. Components plug in by reading or writing it.